

WEEK 2 ASSIGNMENT: GIT AND GIT HUB

Assignment 5: CodeTrack— Initial Git Setup (Local Only)

Task 1 — Create a Local Project Directory

I simulated the creating of a new **CodeTrack** project for a new development.

I created a new directory using the command ~ **mkdir CodeTrack**

I navigated to the new directory using ~ **cd CodeTrack**

I initialized the directory using ~ **git init**. With this, the directory became a **repository** making it possible for git to track

Then, I confirmed the presence of **.git** folder in the directory after initialization using ~ **ls -al**

```
user@Godwin MINGW64 ~  
$ mkdir CodeTrack  
user@Godwin MINGW64 ~  
$ cd CodeTrack  
user@Godwin MINGW64 ~/CodeTrack  
$ git init  
Initialized empty Git repository in C:/Users/user/CodeTrack/.git/  
user@Godwin MINGW64 ~/CodeTrack (main)  
$ touch .gitignore README.md  
user@Godwin MINGW64 ~/CodeTrack (main)  
$ ls -al  
total 36  
drwxr-xr-x 1 user 197609 0 Aug 27 11:33 ./  
drwxr-xr-x 1 user 197609 0 Aug 27 11:28 ../  
drwxr-xr-x 1 user 197609 0 Aug 27 11:32 .git/  
-rw-r--r-- 1 user 197609 0 Aug 27 11:33 .gitignore  
-rw-r--r-- 1 user 197609 0 Aug 27 11:33 README.md  
user@Godwin MINGW64 ~/CodeTrack (main)  
$
```

Task 2 — Configure Git Locally for CodeTrack

Configure identity only for this repo (recommended for team/enterprise scenarios when identities differ by project).

Using this command ~ **git config --local user.name "Your Name"** . I confirmed my identity (name) for local configuration

Using this command ~ **git config --local user.email "Your email address"** . I confirmed my identity (email) for local configuration

I ran ~ **git config --local --list** to confirmed credential configuration

Use local config when a specific repo requires a different identity or behavior than your global defaults. A work project where you must use your company email instead of your personal one. Local overrides global config

```
user@Godwin MINGW64 ~/CodeTrack (main)  
$ git config --local user.name Godwin Williams  
user@Godwin MINGW64 ~/CodeTrack (main)  
$ git config --local user.email eyolegoo@gmail.com  
user@Godwin MINGW64 ~/CodeTrack (main)  
$ git config --local --list  
core.repositoryformatversion=0  
core.filemode=false  
core.bare=false  
core.logallrefupdates=true
```

```
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
user.name=Godwin
user.email=eyolegoo@gmail.com

user@Godwin MINGW64 ~/CodeTrack (main)
$ |
```

Task 3 — Configure Git Globally

Using this command ~ **`git config --global user.name "Your Name"`** . I confirmed my identity (name) for local configuration

Using this command ~ **`git config --global user.email "Your email address"`** . I confirmed my identity (email) for local configuration

I ran ~ **`git config --global --list`** to confirmed credential configuration

With Global, values are set globally when they represent your default identity or setup across most or all repos on your machine. Your main name and email are used for commits

```
user@Godwin MINGW64 ~/CodeTrack (main)
$ git config --global user.name Godwin Williams

user@Godwin MINGW64 ~/CodeTrack (main)
$ git config --global user.email eyolegoo@gmail.com

user@Godwin MINGW64 ~/CodeTrack (main)
$ git config --local --list
core.repositoryformatversion=0
core.filemode=false
core.bare=false
core.logallrefupdates=true
core.symlinks=false
core.ignorecase=true
user.name=Godwin
user.email=eyolegoo@gmail.com

user@Godwin MINGW64 ~/CodeTrack (main)
$ |
```

Assignment 6: Tracking and Staging Changes in a CodeTrack Project

Step 1: Ensure Your Git Setup

I ensured by confirming git is installed using the command ~ **`git --version`** this confirmed git is installed and shows the installed version

Step 2: Navigate to Project Folder

I navigated to the path CodeTrack using ~ **`cd CodeTrack`** and also confirm the current path using ~ **`pwd`**

Step 3: Create and Modify Files

In this directory, I created two files "index.html and style.css" using the command ~ `touch index.html style.css`
Using the ~ `ls` command, I verified these files were created

```
user@Godwin MINGW64 ~  
$ git --version  
git version 2.51.0.windows.1  
  
user@Godwin MINGW64 ~  
$ cd CodeTrack/  
  
user@Godwin MINGW64 ~/CodeTrack (main)  
$ pwd  
/c/Users/user/CodeTrack  
  
user@Godwin MINGW64 ~/CodeTrack (main)  
$ touch index.html style.css  
  
user@Godwin MINGW64 ~/CodeTrack (main)  
$ ls  
index.html  README.md  style.css
```

Using vi editor, I modified the files "index.html and style.css" with the command ~ `vi index.html` and ~ `vi style.css`. This was done one after the other

```
user@Godwin MINGW64 ~/CodeTrack (main)  
$ vi index.html  
  
<!DOCTYPE html>  
<html lang="en">  
  
  <head>  
    <meta charset="UTF-8" />  
    <meta name="viewport" content="width=device-width, initial-scale=1" />  
    <title>Git Assignment - DevOps for Beginners</title>  
    <link rel="stylesheet" href="style.css" />  
  </head>  
  
  <body>  
    <header class="site-header">  
      <div class="header-inner">  
        <h1>Week 2: Git & GitHub Assignment</h1>  
        <p class="cohort-line">  
          Part of <strong>DevOps for Beginners Free Cohort</strong> by  
          <a href="https://www.linkedin.com/in/pravin-mishra-aws-trainer/" target="_blank">Pravin Mishra</a>  
        </p>  
        <p class="playlist-line">  
          If you'd like to learn more, check out the playlist:  
          <a href="https://www.youtube.com/playlist?list=PLV0dqXbCs7bX88JeUzmK4FKTq2hJ5VS89" target="_blank">  
            DevOps for Beginners - Free Playlist</a>  
        </p>  
      </div>  
    </header>  
  
    <main class="container">  
  
      <!-- Student Details -->  
      <section class="card student-info" aria-labelledby="student-details">  
        <h2 id="student-details">Student Details</h2>  
        <p><b>Your Full Name:</b> <span class="placeholder">[Write your name here]</span></p>  
    </main>  
  </body>  
</html>  
-- INSERT --
```

```
user@Godwin MINGW64 ~/CodeTrack (main)  
$ vi style.css  
  
/* -----  
Base  
----- */
```

```

:root {
  --brand-900: #0b3b66;
  /* deep navy header for better contrast */
  --brand-700: #114a80;
  --ink: #1f2937;
  --muted: #6b7280;
  --paper: #ffffff;
  --bg: #f5f7fb;
  --border: #e5e7eb;
  --accent: #2563eb;
  /* link color */
  --accent-ink: #0b3b66;
  --ok: #22c55e;
}

*,
*::before,
*::after {
  box-sizing: border-box;
}

html,
body {
  height: 100%;
  margin: 0;
}

body {
  font-family: "Segoe UI", system-ui, -apple-system, Arial, sans-serif;
  color: var(--ink);
}
style.css [unix] (13:34 27/08/2025)

```

4: Track the Files Using Git

I checked the status of the repository using ~ ***git status***

I staged the untracked files for tracking running the ~ ***git add .*** command. Take note of the dot (.) in the command

I checked the status of the repository using ~ ***git status*** but this time to confirm that the files are in staging area

```

user@Godwin MINGW64 ~/CodeTrack (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        README.md
        index.html
        style.css

nothing added to commit but untracked files present (use "git add" to track)

user@Godwin MINGW64 ~/CodeTrack (main)
$ git add .
warning: in the working copy of 'index.html', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'style.css', LF will be replaced by CRLF the next time Git touches it

user@Godwin MINGW64 ~/CodeTrack (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   README.md
        new file:   index.html
        new file:   style.css

```

Step 5: Commit the Changes

I saved the staged files using ~ ***git commit -m "Initial commit - Added index.html and style.css"***

And verified the commit history using ~ **git log --oneline** or **git log**

```
user@Godwin MINGW64 ~/CodeTrack (main)
$ git commit -m "Initial commit - Added index.html and style.css"
[main (root-commit) b0d16ab] Initial commit - Added index.html and style.css
4 files changed, 305 insertions(+)
create mode 100644 .gitignore
create mode 100644 README.md
create mode 100644 index.html
create mode 100644 style.css

user@Godwin MINGW64 ~/CodeTrack (main)
$ git log --oneline
b0d16ab (HEAD -> main) Initial commit - Added index.html and style.css

user@Godwin MINGW64 ~/CodeTrack (main)
$ git log
commit b0d16ab26824a540e864f55376a6a6ae3ca8767f (HEAD -> main)
Author: Godwin <eyolegoo@gmail.com>
Date:   Wed Aug 27 13:45:56 2025 +0100

    Initial commit - Added index.html and style.css

user@Godwin MINGW64 ~/CodeTrack (main)
$
```

Step 6: Modify a File and Commit Again

I opened the **index.html** file in your favorite browser. (Tip: right-click the file → choose Open With Browser).

Read the instructions written inside **index.html** and follow them step by step to complete the assignment. It must be like below:

Week 2: Git & GitHub Assignment

Part of **DevOps for Beginners Free Cohort** by [Pravin Mishra](#)

If you'd like to learn more, check out the playlist: [DevOps for Beginners – Free Playlist](#)

Student Details

Your Full Name: [Godwin Williams]

Your Group Number: [Group 5]

Concepts You've Learned (Git & GitHub)

Briefly explain the Git & GitHub concepts you've learned so far (like repository basics, status, staging, commits, history, branching, merging, etc.).

- ✦ Imagine you're explaining these ideas to a 10th-grade student, so use **simple words** and **easy-to-understand examples or analogies**.
- ✦ Also, think like a **DevOps/Cloud engineer** – sometimes applications fail, and while you may not fix the app code itself, you need to **look into code snippets, logs, or commit history** to understand what went wrong.
- ✦ For example: a wrong database connection string in code will show up in logs, and using Git history you can check *who changed it and when*.

Your Task:

- Explain Git concepts in your own words (as if teaching someone new).
- Connect at least one Git concept with a real-world debugging scenario. (e.g., "if an application suddenly stops connecting to the database, I can use `git log` to see if someone recently changed the configuration file.")
- Write 8–10 lines in simple language, add screenshots if possible.

✦ **What is Git?**

Git is a version control system tool that helps you track changes in your files (usually code, but it can be anything). It is versioning in its operation because over time this code gets new feature updates that get integrated into the original, making a newer version of the code without breaking the original. With Git, you can move back to an earlier version, see who changed what, and collaborate with others without overwriting each other's work.

♦ Key Concepts

Repository (repo)

A repository is like a project folder that Git is tracking. It contains all your files plus the full history of changes. There are 2 types of repos:

- **Local repo:** The one on your computer.
- **Remote repo:** The one on a server like GitHub or Bitbucket to share with others.

Staging Area

Imagine packing for a trip: you put items into your suitcase before leaving. The staging area is like that suitcase — you choose which changes to include in your next commit. Git tracks these changes when you run:

```
git add .
```

Commit

A commit is like a snapshot of your project at a certain moment. Every time you reach a point you want to save, you make a commit with a message:

```
git commit -m "update message"
```

Over time, commits create the history of your project. Note: Only changes in the staging area are saved.

Branch

A branch is like a parallel timeline for your project. You start on a main branch (often called *main* or *master*). When adding a new feature, it's best to create a branch so you can experiment safely. Later, you can merge it back.

```
git branch new-branch-name
```

Merge

Merge means combining changes from one branch into another. Git usually does this automatically. If two people edit the same line, a **merge conflict** occurs and must be resolved manually.

Remote

A remote is a copy of your repository on the internet (e.g., GitHub, GitLab, Bitbucket). You **push** commits to the remote so others can see them, and **pull** commits from it to stay updated.

Clone

Cloning means making a copy of a remote repository onto your computer so you can start working on it.

♦ Debugging Example

I mistakenly committed an incomplete change with:

```
git commit -m "latest update"
```

To fix it, I updated the file, staged it again:

```
git add .
```

Then amended the last commit:

```
git commit --amend
```

This kept the same commit message but included the corrected changes.

[Write your learning summary here – including both Git basics and one debugging example]

I checked the status of the repository ~ ***git status***

I staged the modified **index.html** file ~ **git add index.html**
I saved the staged file ~ **git commit -m "Meaninfull message"**
I verified the commit history again ~ **git log --oneline**

```
user@Godwin MINGW64 ~/CodeTrack (main)
$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   index.html

no changes added to commit (use "git add" and/or "git commit -a")

user@Godwin MINGW64 ~/CodeTrack (main)
$ git add index.html
warning: in the working copy of 'index.html', LF will be replaced by CRLF the next time Git touches it

user@Godwin MINGW64 ~/CodeTrack (main)
$ git commit -m "update made on index.html file"
[main 753d8ed] update made on index.html file
1 file changed, 99 insertions(+), 2 deletions(-)

user@Godwin MINGW64 ~/CodeTrack (main)
$ git log --oneline
753d8ed (HEAD -> main) update made on index.html file
b0d16ab Initial commit - Added index.html and style.css

user@Godwin MINGW64 ~/CodeTrack (main)
$
```

Step 7: Deploy this application on EC2 Instance

I launched an EC2 Instance, connected to the EC2 Instance via ssh on the terminal using ~ `ssh -i key.pem ec2-user@<EC2-Public-IP>`

```
user@Gadwin MINGW64 ~$ cd Downloads

user@Gadwin MINGW64 ~/Downloads$ chmod 400 "linuxdemokey.pem"

user@Gadwin MINGW64 ~/Downloads$ ssh -i linuxdemokey.pem ec2-user@ec2-167-171-148-166.eu-north-1.compute.amazonaws.com
The authenticity of host 'ec2-167-171-148-166.eu-north-1.compute.amazonaws.com (167.171.148.166)' can't be established.
ED25519 key fingerprint is SHA256:gndZrXtQwoqfMw1HbJdFLrnbnAzoym6XfmB3LVkbQ.
This host key is known by the following other names/addresses:
- .ssh/knwon-hosts/7fb2-ec2-167-168-101-39.eu-north-1.compute.amazonaws.com
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-167-171-148-166.eu-north-1.compute.amazonaws.com' (ED25519) to the list of known hosts.
```

The screenshot shows a terminal window titled 'Gadwin'. The user navigates to the 'Downloads' directory and sets permissions on a file named 'linuxdemokey.pem'. Then, they attempt an SSH connection to an EC2 instance at IP address 167.171.148.166. The terminal output shows the standard SSH warning about the host's authenticity, followed by the user pressing 'yes' to accept the fingerprint. The connection is successful, displaying the Amazon Linux 2023 logo and banner, which includes the URL 'https://aws.amazon.com/linux/amazon-linux-2023'. At the bottom, it shows the last login time as Mon Aug 26 12:38:35 2025 from IP 102.89.22.92.

I updated Packages using ~ ***sodu yum update***
 Installed Nginx ~ ***sodu yum install nginx -y***
 I made it start automatically after reboot using ~ ***sudo systemctl enable nginx***
 Started the Nginx Service ~ ***sudo systemctl start nginx***
 And finally, I confirm it is up using ~ ***sudo systemctl status nginx***

```
[ec2-user@ip-172-31-47-15 ~]$ pwd
/home/ec2-user
[ec2-user@ip-172-31-47-15 ~]$ sudo yum update
Last metadata expiration check: 0:13:11 ago on Wed Aug 27 20:33:36 2025.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-172-31-47-15 ~]$ sudo yum install nginx -y
Last metadata expiration check: 0:13:11 ago on Wed Aug 27 20:33:36 2025.
```

```
Last metadata expiration check: 0:13:35 ago on Wed Aug 27 20:33:36 2025.
Dependencies resolved.

Package Architecture Version Repository Size
-----
Installing:
nginx x86_64 1:1.28.0-1.amzn2023.0.2 amazonlinux 33 k
Installing dependencies:
generic-logos-httpd noarch 18.0-0-12.amzn2023.0.3 amazonlinux 19 k
gnupgtools-libs x86_64 2.9.2-1.amzn2023.0.3 amazonlinux 388 k
libunwind x86_64 1.4.0-5.amzn2023.0.2 amazonlinux 66 k
nginx-core x86_64 1:1.28.0-1.amzn2023.0.2 amazonlinux 686 k
nginxfilesystem noarch 1:1.28.0-1.amzn2023.0.2 amazonlinux 9.6 k
nginx-mimeetypes noarch 2.1.49-3.amzn2023.0.3 amazonlinux 21 k

Transaction Summary
-----
Install 7 Packages

Total download size: 1.1 M
Installed size: 3.7 M
Downloading Packages:
(1/7): generic-logos-httpd-18.0-0-12.amzn2023.0.3.noarch.rpm 563 kB/s | 19 kB 00:00
(2/7): libunwind-1.4.0-5.amzn2023.0.2.x86_64.rpm 1.7 MB/s | 66 kB 00:00
(3/7): gnupgtools-libs-2.9.2-1.amzn2023.0.3.x86_64.rpm 6.1 MB/s | 388 kB 00:00
(4/7): nginx-1.28.0-1.amzn2023.0.2.x86_64.rpm 1.6 MB/s | 33 kB 00:00
(5/7): nginxfilesystem-1.28.0-1.amzn2023.0.2.noarch.rpm 467 kB/s | 9.6 kB 00:00
(6/7): nginx-core-1.28.0-1.amzn2023.0.2.x86_64.rpm 17 MB/s | 686 kB 00:00
[ec2-user@ip-172-31-47-15 ~]$ sudo systemctl enable nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
[ec2-user@ip-172-31-47-15 ~]$ sudo systemctl start nginx
[ec2-user@ip-172-31-47-15 ~]$ sudo systemctl status nginx
* nginx.service - The nginx HTTP and reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: disabled)
   Active: active (running) since Wed 2025-08-27 20:48:50 UTC; 18s ago
     Process: 3135 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
     Process: 3136 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
     Process: 3137 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
    Main PID: 3138 (nginx)
       Tasks: 3 (limit 1014)
      Memory: 3.2M
         CPU: 55ms
      CGroup: /system.slice/nginx.service
              └─3138 "nginx: master process /usr/sbin/nginx"
                  └─3139 "nginx: worker process"
                      └─3140 "nginx: worker process"

Aug 27 20:48:50 ip-172-31-47-15.eu-north-1.compute.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Aug 27 20:48:50 ip-172-31-47-15.eu-north-1.compute.internal nginx[3136]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Aug 27 20:48:50 ip-172-31-47-15.eu-north-1.compute.internal nginx[3136]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Aug 27 20:48:50 ip-172-31-47-15.eu-north-1.compute.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
[ec2-user@ip-172-31-47-15 ~]$
```

I deployed the applications (**index.html** and **style.css**) to my EC2 Instance from my local system

To move the applications, I used ~ **scp -i key.pem applications ec2-user@remote Public IP: /destination folder path**

I logged into my EC2 Instance and confirm the files were moved successfully. The files now sit in **/home/ec2-user/** since **/home/ec2-user/** is the destination path

The plan is to move these files to **/usr/share/nginx/html**

```
user@Godwin MINGW64 ~
$ cd Downloads

user@Godwin MINGW64 ~/Downloads
$ scp -i linuxdemokey.pem index.html style.css ec2-user@16.171.148.166:/home/ec2-user
The authenticity of host '16.171.148.166 (16.171.148.166)' can't be established.
ED25519 key fingerprint is SHA256:snidZrIGTuoqIMw1Hb4jFLrwnbnAzoym6X+mB3LVKbQ.
This host key is known by the following other names/addresses:
  ~/.ssh/known_hosts:378: ec2-13-60-181-37.eu-north-1.compute.amazonaws.com
  ~/.ssh/known_hosts:380: ec2-16-171-148-166.eu-north-1.compute.amazonaws.com
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '16.171.148.166' (ED25519) to the list of known hosts.
index.html 100% 7196 34.1KB/s 00:00
style.css 100% 4180 22.2KB/s 00:00

user@Godwin MINGW64 ~/Downloads
$ |
[ec2-user@ip-172-31-47-15 ~]$ pwd
/home/ec2-user
[ec2-user@ip-172-31-47-15 ~]$ ls -la
total 40
drwxr-xr-x 3 ec2-user ec2-user 160 Aug 27 21:02 .
drwxr-xr-x 3 root root 22 Aug 25 07:46 ..
-rw-r--r-- 1 ec2-user ec2-user 801 Aug 28 14:52 .bash_history
-rw-r--r-- 1 ec2-user ec2-user 19 Jan 28 2023 .bash_logout
-rw-r--r-- 1 ec2-user ec2-user 141 Jan 28 2023 .bash_profile
-rw-r--r-- 1 ec2-user ec2-user 492 Jan 28 2023 .bashrc
-rw-r--r-- 1 ec2-user ec2-user 29 Aug 25 09:26 .gitconfig
-rw-r--r-- 1 ec2-user ec2-user 20 Aug 25 11:22 .lesshst
drwxr-xr-x 2 ec2-user ec2-user 29 Aug 25 07:46 .ssh
-rw-r--r-- 1 ec2-user ec2-user 7196 Aug 27 21:02 index.html
-rw-r--r-- 1 ec2-user ec2-user 4180 Aug 27 21:02 style.css
[ec2-user@ip-172-31-47-15 ~]$
```

I navigated to **/usr/share/nginx/html** using ~ **cd /usr/share/nginx/html** and **deleted** the **index.html** file in html folder of nginx. Nginx came with this file

Using ~ **sudo rm index.html**, I deleted the **index.html** in **/usr/share/nginx/html**. Using ~ **ls -al**, I confirmed the file has been deleted. The

Using `ls -la /usr/share/nginx/html`, I deleted the `index.html` in `/usr/share/nginx/html` using `rm index.html`. I confirmed the file has been deleted. The idea is to replace this deleted file with the new application files in the `/home/ec2-user/`. The next step is to move the application files (`index.html` and `style.css`) in `/home/ec2-user` into `/usr/share/nginx/html`

```
[ec2-user@ip-172-31-47-15 ~]$ cd /usr/share/nginx/html
[ec2-user@ip-172-31-47-15 html]$ ls -la
total 16
drwxr-xr-x. 3 root root 112 Aug 27 20:47
drwxr-xr-x. 4 root root 33 Aug 27 20:47
-rw-r--r--. 1 root root 3650 Aug 12 21:18 404.html
-rw-r--r--. 1 root root 3693 Aug 12 21:18 50x.html
drwxr-xr-x. 2 root root 27 Aug 27 20:47 icons
-rw-r--r--. 1 root root 615 Aug 12 21:19 index.html
-rw-r--r--. 1 root root 368 Aug 12 21:18 nginx-logo.png
lrwxrwxrwx. 1 root root 14 Aug 12 21:19 poweraby.png -> nginx-logo.png
[ec2-user@ip-172-31-47-15 html]$ sudo rm index.html
[ec2-user@ip-172-31-47-15 html]$ ls -la
total 12
drwxr-xr-x. 3 root root 94 Aug 27 21:10
drwxr-xr-x. 4 root root 33 Aug 27 20:47
-rw-r--r--. 1 root root 3650 Aug 12 21:18 404.html
-rw-r--r--. 1 root root 3693 Aug 12 21:18 50x.html
drwxr-xr-x. 2 root root 27 Aug 27 20:47 icons
-rw-r--r--. 1 root root 368 Aug 12 21:18 nginx-logo.png
lrwxrwxrwx. 1 root root 14 Aug 12 21:19 poweraby.png -> nginx-logo.png
[ec2-user@ip-172-31-47-15 html]$
```

Using `cd` I navigated to `/home/ec2-user/`

In `/home/ec2-user/` I moved the files using `sudo mv index.html style.css /usr/share/nginx/html`

I used `ls -la` in `/home/ec2-user/` to confirm the files were moved successfully

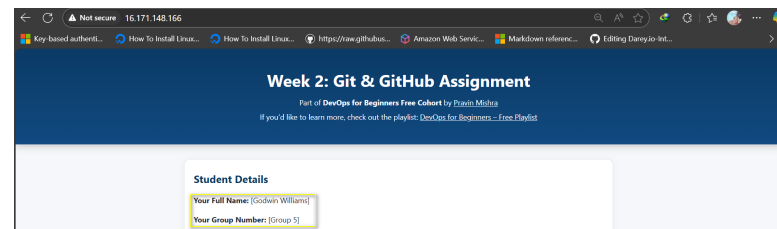
Then using `cd /usr/share/nginx/html` I navigated to `/usr/share/nginx/html`. And using `ls -la` confirmed the new files (`index.html` and `style.css`) are present

```
[ec2-user@ip-172-31-47-15 html]$ cd
[ec2-user@ip-172-31-47-15 ~]$ pwd
/home/ec2-user
[ec2-user@ip-172-31-47-15 ~]$ ls -la
total 40
drwxr-xr-x. 3 ec2-user ec2-user 164 Aug 27 21:02
drwxr-xr-x. 3 root root 112 Aug 27 20:47
-rw-r--r--. 1 ec2-user ec2-user 801 Aug 25 14:52 .bash_history
-rw-r--r--. 1 ec2-user ec2-user 18 Jan 28 2023 .bash_logout
-rw-r--r--. 1 ec2-user ec2-user 178 Jan 28 2023 .bash_profile
-rw-r--r--. 1 ec2-user ec2-user 492 Jan 28 2023 .bashrc
-rw-r--r--. 1 ec2-user ec2-user 29 Aug 25 09:26 gitconfig
-rw-r--r--. 1 ec2-user ec2-user 28 Aug 25 11:22 .ssh
drwxr-xr-x. 2 ec2-user ec2-user 29 Aug 25 07:46 .ssh
-rw-r--r--. 1 ec2-user ec2-user 1396 Aug 27 21:02 index.html
-rw-r--r--. 1 ec2-user ec2-user 4188 Aug 27 21:02 style.css
[ec2-user@ip-172-31-47-15 ~]$ mv index.html style.css /usr/share/nginx/html
[ec2-user@ip-172-31-47-15 ~]$ ls -la
total 40
drwxr-xr-x. 3 ec2-user ec2-user 129 Aug 27 21:14
drwxr-xr-x. 3 root root 112 Aug 27 20:47
-rw-r--r--. 1 ec2-user ec2-user 801 Aug 25 14:52 .bash_history
-rw-r--r--. 1 ec2-user ec2-user 18 Jan 28 2023 .bash_logout
-rw-r--r--. 1 ec2-user ec2-user 141 Jan 28 2023 .bash_profile
-rw-r--r--. 1 ec2-user ec2-user 492 Jan 28 2023 .bashrc
-rw-r--r--. 1 ec2-user ec2-user 29 Aug 25 09:26 gitconfig
-rw-r--r--. 1 ec2-user ec2-user 28 Aug 25 11:22 .ssh
drwxr-xr-x. 2 ec2-user ec2-user 29 Aug 25 07:46 .ssh
[ec2-user@ip-172-31-47-15 ~]$ cd /usr/share/nginx/html
[ec2-user@ip-172-31-47-15 html]$ pwd
/usr/share/nginx/html
[ec2-user@ip-172-31-47-15 html]$ ls -la
total 20
drwxr-xr-x. 3 root root 129 Aug 27 21:14
drwxr-xr-x. 4 root root 33 Aug 27 20:47
-rw-r--r--. 1 root root 3650 Aug 12 21:18 404.html
-rw-r--r--. 1 root root 3693 Aug 12 21:18 50x.html
drwxr-xr-x. 2 root root 27 Aug 27 20:47 icons
-rw-r--r--. 1 ec2-user ec2-user 1396 Aug 27 21:02 index.html
-rw-r--r--. 1 root root 368 Aug 12 21:18 nginx-logo.png
lrwxrwxrwx. 1 root root 14 Aug 12 21:19 poweraby.png -> nginx-logo.png
-rw-r--r--. 1 ec2-user ec2-user 4188 Aug 27 21:02 style.css
[ec2-user@ip-172-31-47-15 html]$
```

Time to access our application

On a web browser, I entered the Public IP of the instance: **16.171.148.166**

The application ran successfully



Concepts You've Learned (Git & GitHub)

Briefly explain the Git & GitHub concepts you've learned so far (like repository basics, status, staging, commits, history, branching, merging, etc.).

- Imagine you're explaining these ideas to a 10th-grade student, so use simple words and easy-to-understand examples or analogies.

Not secure 16.171.148.166

Key-based authenti... How to Install Linux... How to Install Linux... https://raw.githubusercontent... Amazon Web Servic... Markdown reference... Editing Darcy.io int...

Your Task:

- Explain Git concepts in your own words (as if teaching someone new).
- Connect at least one Git concept with a real-world debugging scenario (e.g., "If an application suddenly stops connecting to the database, I can use `git` to see if someone recently changed the configuration file.")
- Write 8-10 lines in simple language, add screenshots if possible.

What is Git?

Git is a version control system tool that helps you track changes in your files (usually code, but it can be anything). It is versioning in its operation because over time this code gets new feature updates that get integrated into the original, making a newer version of the code without breaking the original. With Git, you can move back to an earlier version, see who changed what, and collaborate with others without overwriting each other's work.

Key Concepts

Repository (repo)

A repository is like a project folder that Git is tracking. It contains all your files plus the full history of changes. There are 2 types of repos:

- Local repo:** The one on your computer.
- Remote repo:** The one on a server like GitHub or Bitbucket to share with others.

Staging Area

Imagine packing for a trip: you put items into your suitcase before leaving. The staging area is like that suitcase — you choose which changes to include in your next commit. Git tracks these changes when you run:

git add -

Commit

A commit is like a snapshot of your project at a certain moment. Every time you reach a point you want to save, you make a commit with a message:

```
git commit -m "update message"
```

Over time, commits create the history of your project. Note: Only changes in the staging area are saved.

Branch

A branch is like a parallel timeline for your project. You start on a main branch (often called `main` or `master`). When adding a new feature, it's best to create a branch so you can experiment safely. Later, you can merge it back.

```
git branch new-branch-name
```

Merge

Merge means combining changes from one branch into another. Git usually does this automatically. If two people edit the same line, a **merge conflict** occurs and must be resolved manually.

Remote

A remote is a copy of your repository on the internet (e.g., GitHub, GitLab, Bitbucket). You **push** commits to the remote so others can see them, and **pull** commits from it to stay updated.

Clone

Cloning means making a copy of a remote repository onto your computer so you can start working on it.

Not secure 16.171.148.166

Key-based authenti... How to Install Linux... How to Install Linux... https://raw.githubusercontent... Amazon Web Servic... Markdown reference... Editing Darcy.io int...

Remote

A remote is a copy of your repository on the internet (e.g., GitHub, GitLab, Bitbucket). You **push** commits to the remote so others can see them, and **pull** commits from it to stay updated.

Clone

Cloning means making a copy of a remote repository onto your computer so you can start working on it.

Debugging Example

I mistakenly committed an incomplete change with:

```
git commit -m "latest update"
```

To fix it, I updated the file, staged it again:

```
git add -
```

Then amended the last commit:

```
git commit --amend
```

This kept the same commit message but included the corrected changes.
[Write your learning summary here — including both Git basics and one debugging example]

© 2025 DevOps for Beginners • [View Details](#)

Assignment 7: Branching Workflow — Add & Verify a Contact Page

On my local system, I navigated to CodeTrack folder using ~ `cd CodeTrack`

To confirm the current state of the repository, I ran ~ `git status`

To confirm the current branch of the repository, I ran ~ `git branch`

Step 1 — Create and switch to a feature branch

I created a new branch using ~ `git checkout -b feature/contact-page`

I ran ~ `git branch` to confirm I'm on the newly created branch

Step 2 — Add contact.html (in the branch)

On the new branch, I created a file contact.html using ~ `touch contact.html`

I opened the file using an editor with the command ~ `vi contact.html` and added some content, saved and exit

```
<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <title>Contact - CodeTrack</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Contact Us</h1>
  <p>Email: mail@pravinmishra.in</p>
  <p>Website: https://thecloudadvisory.com/</p>
</body>
</html>
```

```
user@Godwin-MINGW64 ~
$ cd CodeTrack/
user@Godwin-MINGW64 ~/CodeTrack (main)
$ pwd
/C:/Users/user/CodeTrack
user@Godwin-MINGW64 ~/CodeTrack (main)
$ git status
On branch main
nothing to commit, working tree clean
user@Godwin-MINGW64 ~/CodeTrack (main)
$ git branch
* main
user@Godwin-MINGW64 ~/CodeTrack (main)
$ git checkout -b feature/contact-page
Switched to a new branch 'feature/contact-page'
user@Godwin-MINGW64 ~/CodeTrack (feature/contact-page)
$ git branch
  feature/contact-page
* main
user@Godwin-MINGW64 ~/CodeTrack (feature/contact-page)
$ touch contact.html
user@Godwin-MINGW64 ~/CodeTrack (feature/contact-page)
$ vi contact.html

<!doctype html>
<html>
<head>
  <meta charset="utf-8">
  <title>Contact - CodeTrack</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <h1>Contact Us</h1>
```

```

<p>Email: mail@pravinmishra.in</p>
<p>Website: https://thecloudadvisory.com/</p>
</body>
</html>
~
~
~
~
~

```

I staged the contact.html file using ~ **git add contact.html**

Then saved the staged file using ~ **git commit -m "feat(contact): add contact page with email and phone"**

```

user@Godwin MINGW64 ~/CodeTrack (feature/contact-page)
$ git add contact.html
warning: in the working copy of 'contact.html', LF will be replaced by CRLF the next time Git touches it

user@Godwin MINGW64 ~/CodeTrack (feature/contact-page)
$ git commit -m "feat(contact): add contact page with email and phone"
[feature/contact-page 0d88342] feat(contact): add contact page with email and phone
1 file changed, 13 insertions(+)
create mode 100644 contact.html

user@Godwin MINGW64 ~/CodeTrack (feature/contact-page)
$ |

```

Step 3 — Add a link to the contact page in index.html (still on the branch)

I Inserted a new paragraph directly below the existing playlist paragraph (the one with class playlist-line):

```

<p class="playlist-line">
  If you'd like to learn more, check out the playlist:
  <a href="https://www.youtube.com/playlist?list=PLVOdqXbCs7bX88JeUZmK4fKTq2hJ5VS89" target="_blank">
  >
    DevOps for Beginners – Free Playlist
  </a>
</p>

<!-- ADD THIS NEW PARAGRAPH RIGHT BELOW THE ONE ABOVE -->
<p class="playlist-line">
  Want to reach us? Visit the
  <a href="contact.html">Contact Page</a>.
</p>

```

```

user@Godwin MINGW64 ~/CodeTrack (feature/contact-page)
$ ls -al
total 5/
drwxr-xr-x 1 user 197609  0 Aug 28 00:57 ./
drwxr-xr-x 1 user 197609  0 Aug 28 00:57 ../
drwxr-xr-x 1 user 197609  0 Aug 28 00:13 .git/
-rw-r--r-- 1 user 197609  0 Aug 27 11:33 .gitignore
-rw-r--r-- 1 user 197609 274 Aug 28 00:11 contact.html
-rw-r--r-- 1 user 197609 7196 Aug 27 16:16 index.html
-rw-r--r-- 1 user 197609  0 Aug 27 11:33 README.md
-rw-r--r-- 1 user 197609 4180 Aug 27 13:34 style.css

user@Godwin MINGW64 ~/CodeTrack (feature/contact-page)
$ vi index.html

```

```

# vi index.html

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <title>Git Assignment - DevOps for Beginners</title>
  <link rel="stylesheet" href="style.css" />
</head>

<body>
  <header class="site-header">
    <div class="header-inner">
      <div>Week 2: Git & GitHub Assignment</div>
      <div class="cohort-line">
        Part of <strong>DevOps for Beginners Free Cohort</strong> by
        <a href="https://www.linkedin.com/in/pravin-mishra-aws-trainer/" target="_blank">Pravin Mishra</a>
      </div>
      <div class="playlist-line">
        If you'd like to learn more, check out the playlist:
        <a href="https://www.youtube.com/playlist?list=PLV0dqXbc57bX883eUzmW4FqTq2hJ5VS89" target="_blank">
          DevOps for Beginners - Free Playlist</a>
      </div>
      <div class="playlist-line">
        Want to reach us? Visit the
        <a href="contact.html">Contact Page</a>.
      </div>
    </div>
  </header>

  <main class="container">

    <!-- Student Details -->
    <section class="card student-info" aria-labelledby="student-details">
      <div id="student-details">Student Details</div>
      <div><div>Your Full Name:</div> <span class="placeholder">[Godwin Williams]</span></div>
    </section>
  </main>

```

I staged the newly edited file using ~ **git add index.html**
 then saved using ~ **git commit -m "feat(nav): add Contact Page link to index.html"**

```

user@Godwin MINGW64 ~/CodeTrack (feature/contact-page)
$ vi index.html

user@Godwin MINGW64 ~/CodeTrack (feature/contact-page)
$ git add index.html
warning: in the working copy of 'index.html', LF will be replaced by CRLF the next time Git touches it

user@Godwin MINGW64 ~/CodeTrack (feature/contact-page)
$ git commit -m "feat(nav): add Contact Page link to index.html"
[feature/contact-page d4d5a9b] feat(nav): add Contact Page link to index.html
1 file changed, 6 insertions(+), 1 deletion(-)

user@Godwin MINGW64 ~/CodeTrack (feature/contact-page)
$

```

Step 4 — Verify isolation (switch back to main)

I moved to the main branch using ~ **git checkout main**
 Used ~ **ls** to list items in this branch
contact.html should not be in main yet. Because **contact.html** was worked on the **feature/contact-page** branch

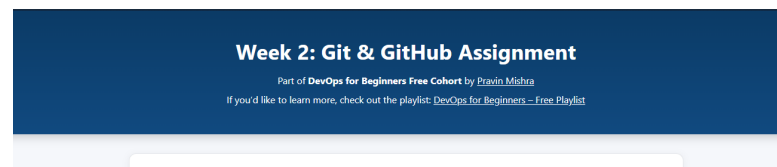
```

user@Godwin MINGW64 ~/CodeTrack (feature/contact-page)
$ git checkout main
Switched to branch 'main'

user@Godwin MINGW64 ~/CodeTrack (main)
$ ls
index.html  README.md  style.css

```

Open **index.html** in your browser → the **Contact Page** link should not exist on main yet.



Student Details

Your Full Name: [Godwin Williams]

Your Group Number: [Group 5]

Concepts You've Learned (Git & GitHub)

Briefly explain the Git & GitHub concepts you've learned so far (like repository basics, status, staging, commits, history, branching, merging, etc.).

- Imagine you're explaining these ideas to a 10th-grade student, so use **simple words** and **easy-to-understand examples or analogies**.
- Also, think like a **DevOps/Cloud engineer** – sometimes applications fail, and while you may not fix the app code itself, you need to **look into code snippets, logs, or commit history** to understand what went wrong.
- For example: a wrong database connection string in code will show up in logs, and using Git history you can check who changed it and when.

Your Task:

- Explain Git concepts in your own words (as if teaching someone new).

Now on the main branch, I merged the feature/contact-page branch to the main using ~ **git merge feature/contact-page**
Using ~ **ls** → I verified **contact.html** is present.

```

user@Godwin MINGW64 ~/CodeTrack (main)
$ git merge feature/contact-page
Updating 753d8ed..d4d5a9b
Fast-forward
 contact.html | 13 ++++++++
 index.html   | 7 +++++
 2 files changed, 19 insertions(+), 1 deletion(-)
 create mode 100644 contact.html

user@Godwin MINGW64 ~/CodeTrack (main)
$ ls
contact.html index.html README.md style.css

user@Godwin MINGW64 ~/CodeTrack (main)
$

```

Open **index.html** in your browser → you should see the **Contact Page** link.

Week 2: Git & GitHub Assignment

Part of DevOps for Beginners Free Cohort by [Pavlin Mishra](#)

If you'd like to learn more, check out the playlist: [DevOps for Beginners – Free Playlist](#)

[Want to reach us? Visit the Contact Page.](#)

Student Details

Your Full Name: [Godwin Williams]

Your Group Number: [Group 5]

Concepts You've Learned (Git & GitHub)

Briefly explain the Git & GitHub concepts you've learned so far (like repository basics, status, staging, commits, history, branching, merging, etc.).

- Imagine you're explaining these ideas to a 10th-grade student, so use **simple words** and **easy-to-understand examples or analogies**.
- Also, think like a **DevOps/Cloud engineer** – sometimes applications fail, and while you may not fix the app code itself, you need to **look into code snippets, logs, or commit history** to understand what went wrong.
- For example: a wrong database connection string in code will show up in logs, and using Git history you can check who changed it and when.

Your Task:

Click the link → it should open **contact.html**.



Contact Us

Email: mail@pravinmishra.in

Website: <https://thecloudadvisory.com/>

Step 6 — Inspect history (nice graph)

For better visualization of history I ran ~ `git log --online --graph --decorate --all`. This command gives you a clear, summarized visualization of the commit history across all branches, helping you understand how branches diverged, where merges happened, and where the current branch stands.

I deleted the feature/contact-page branch using ~ `git branch -d feature/contact-page`

```
user@Godwin-MINGW64 ~/CodeTrack (main)
$ git log --online --graph --decorate --all
* d4d5a9b (HEAD -> main, feature/contact-page) feat(nav): add Contact Page link to index.html
* 0d88342 feat(contact): add contact page with email and phone
* 753d8ed update made on index.html file
* b0d16ab Initial commit - Added index.html and style.css

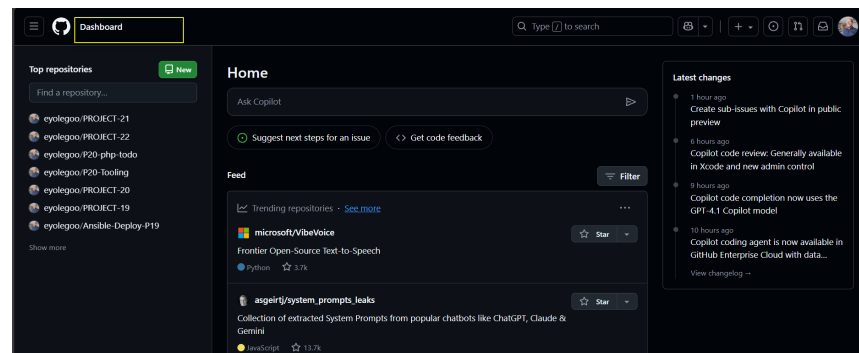
user@Godwin-MINGW64 ~/CodeTrack (main)
$ git branch -d feature/contact-page
Deleted branch feature/contact-page (was d4d5a9b).

user@Godwin-MINGW64 ~/CodeTrack (main)
$
```

Assignment 8: Setting Up GitHub for CodeTrack

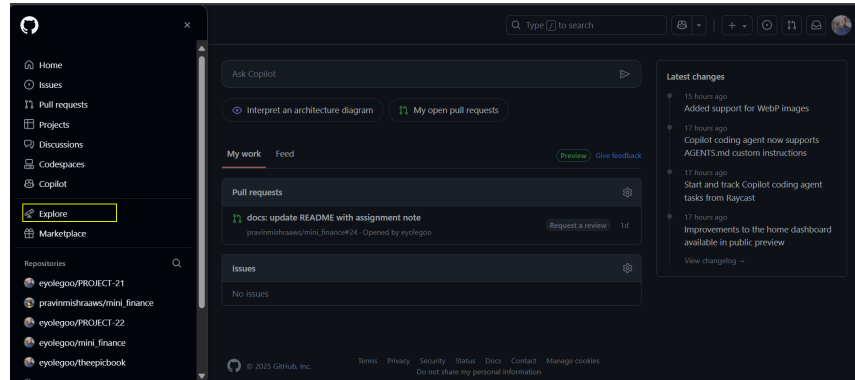
Step 1 — Create a GitHub Account

I have an existing GitHub account

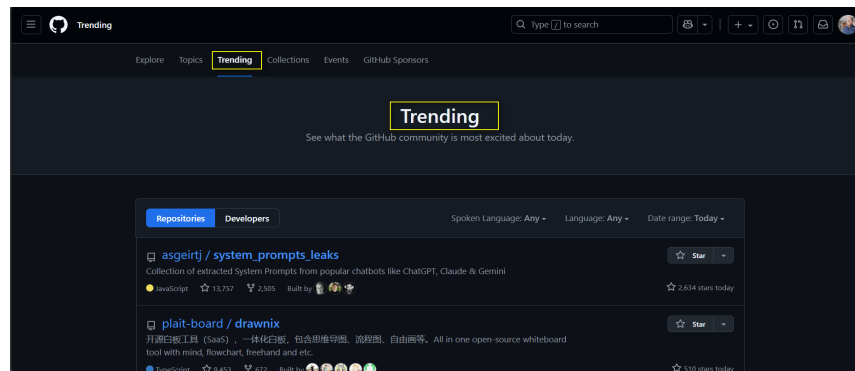


Step 2 — Explore GitHub Features

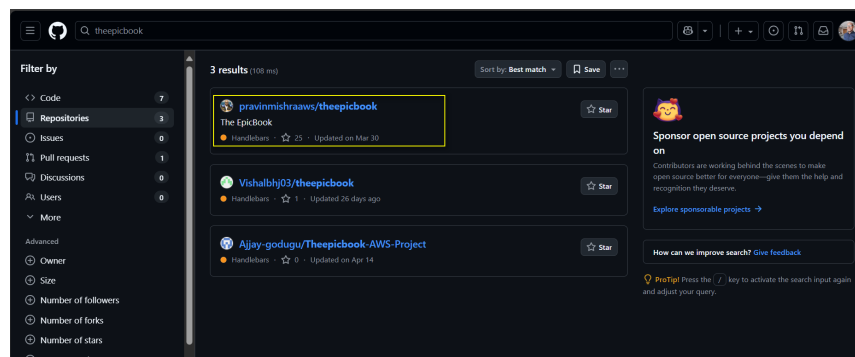
From the top menu, I clicked Explore.



I browsed Trending Repositories to see what's popular.

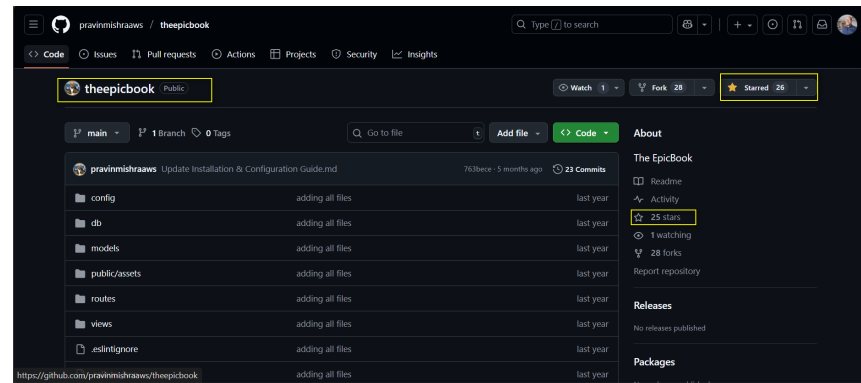


Using the search bar, I searched for an open-source project (e.g., type theepicbook).

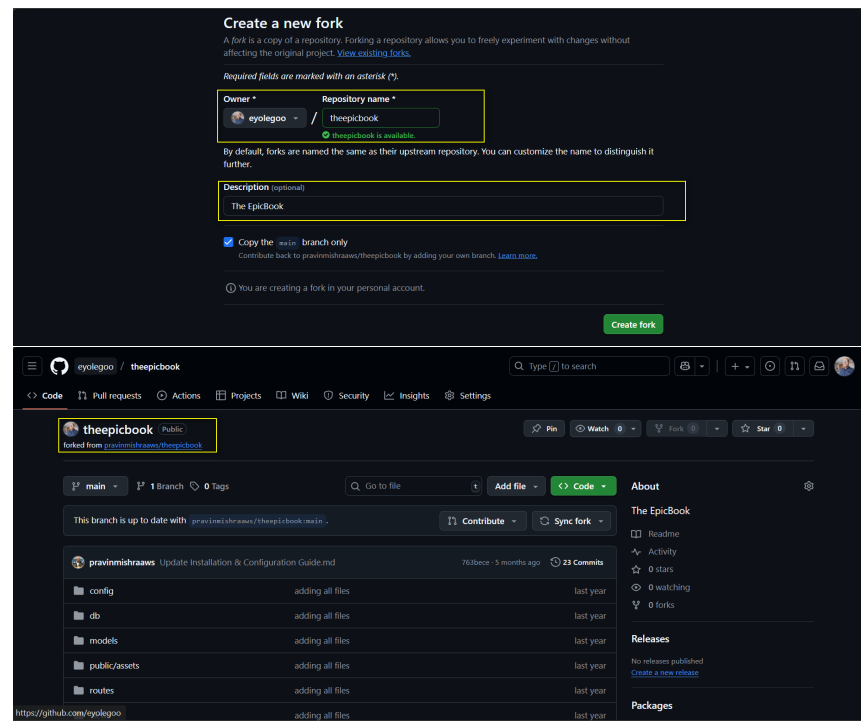




I clicked the Star button to show my interest on this repository



Finally, I forked the repository to create my own copy



Step 3 — Update Your GitHub Profile

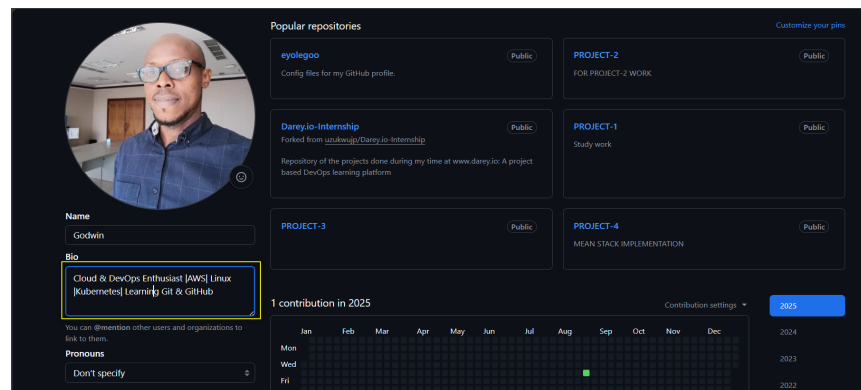
I clicked the profile picture (top-right) → Your Profile.

I clicked on edit Profile and:

Added a short bio (e.g., "Cloud & DevOps Enthusiast |AWS|Linux|Kubernetes|Learning Git & GitHub").

Optionally add location, company/school, and social links.

I uploaded a profile picture (optional but recommended).



Why is it important to have a professional GitHub profile as a developer?

A professional GitHub profile showcases your coding skills, project experience, and technical growth in a way that resumes alone can't. It acts as a living portfolio, giving recruiters and collaborators direct insight into how you write, structure, and maintain code. It also signals your engagement with the developer community through contributions, issue tracking, and open-source involvement. Ultimately, it builds credibility and opens doors to opportunities that value transparency and proven ability.

Assignment 9: Collaborating on Mini-Finance with GitHub

Step 0 — Access Existing Mini-Finance Code

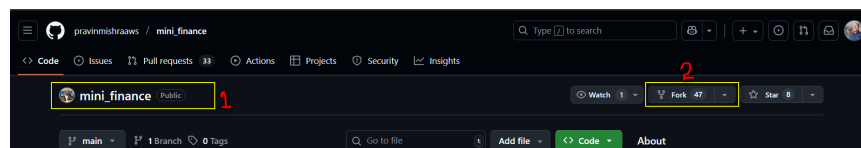
The upstream repository exists at:

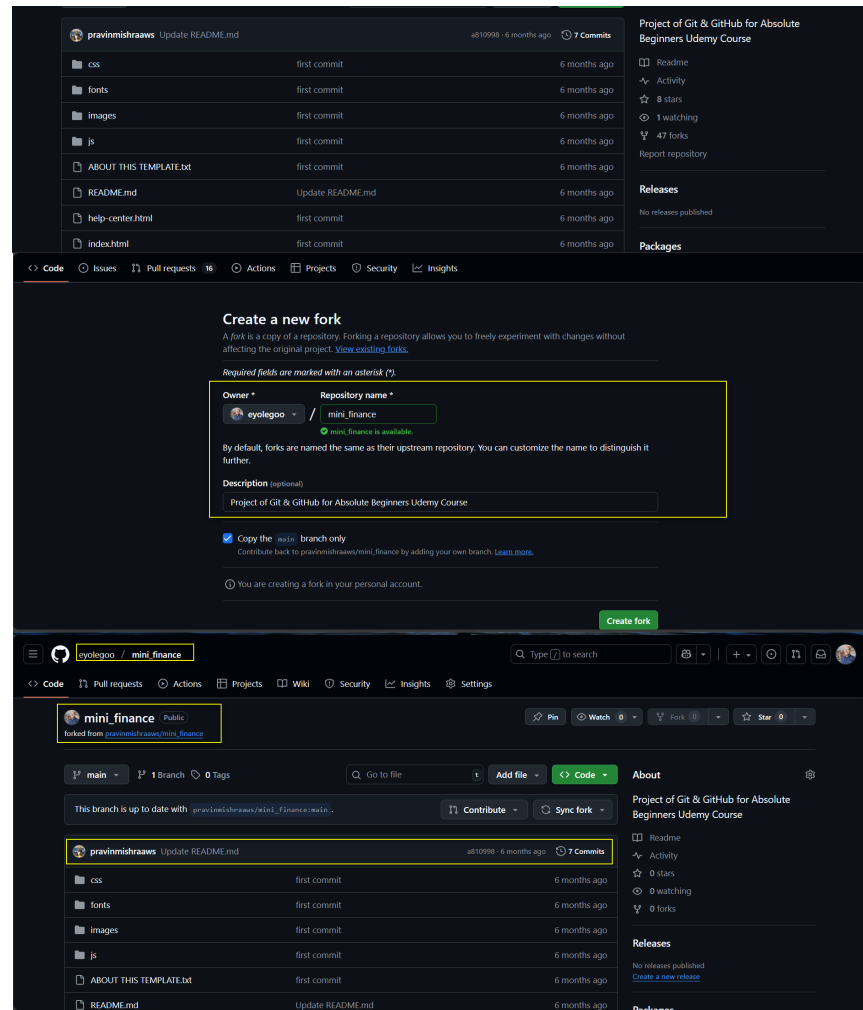
https://github.com/pravinmishraaws/mini_finance

I forked this repository to my GitHub account, clone, and work with it.

Step 1 — Fork & Authenticate

I logged into my GitHub and forked the mini_finance repository into my account.





On my local system terminal, I configured authentication using the SSH method: SSH provides a secure way of authentication without entering the credentials each time

I navigated to the **CodeTrack** folder (repo)

With this command ~ **ssh-keygen -t rsa -b 4096 -C "your-email@example.com"**

These generated keys (private and public) are saved in the default location (~/.ssh/id_rsa).

I copied the public key using ~ **cat ~/.ssh/id_rsa.pub** . The purpose is to paste it in GitHub

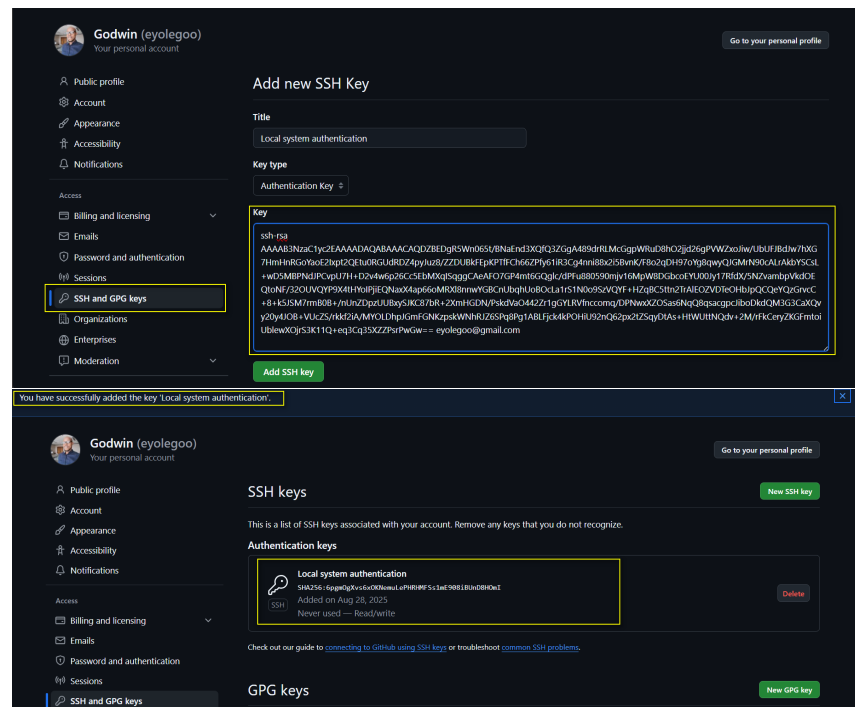
```
user@codin MINGW64 ~/CodeTrack (main)
$ ssh-keygen -t rsa -b 4096 -C "eyolegoo@gmail.com"
Generating public/private rsa key pair.
Enter file in which to save the key (/c:/Users/user/.ssh/id_rsa):
Enter passphrase for "/c:/Users/user/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
```

```
Enter same passphrase again:
Your identification has been saved in /c/Users/user/.ssh/id_rsa
Your public key has been saved in /c/Users/user/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:6gm0gYx6d0lleuLcPHRWHF5s1mE908iBUnD0H0mI eyolegoo@gmail.com
The key's randomart image is:
+--[RSA 4096]-----+
| .oXo. |
| oo+Bl |
| =o.o.. |
| . Eo+ o . |
| o... .So |
| . o. . |
| [o+ . + |
| ++m. = |
| +DAB= |
| -----[SHA256]-----+

user@Godwin MINGW64 ~/CodeTrack (main)
$ cat ~/.ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCAQDZBEGRSWh06S4/8NaEnd3XQF3ZGgA489dRMLMcGpWruD8H0ZjJd26gPVWZxo3lw/UBUfJ8dJw7hXG/HmHnRGoYaoE2ixptZQEtU8Gd
rRDZ4pyJuz8/ZZDUBkFepPTFFCh66ZPfY61R3CgInn188x215BvmK/F8o2dH97oYgBqyQJGfW98cAlrAbbYScL+WD5MBPNdJPCpU7HfDZv4W6p26Cc5EWMXqISgggCAeAF07GP4mt
6G6qle4dFueB889mYj18p8p8D0ceceY0R0Jy17R4dK/SN2vambpW4e02t0NF/S32DUWQYp9tHt01Pj1EQnaxX4ap6deMRXl8mmyGBCrU0qH0e0cL1rS1N0u9S2zQVf+H2q6CSctn2
T3ALeOZVDt0HbJpQcQYQzGvcC+84K5JS7zMB88+/nJnZDpU08xyS3Xc87b6+2XmhSDN/PskdYa0u4ZZz1gGyLRVfnc0mq/DPNwXZ0SasNq08qscgpc3ib0dKQm3G3caXQvy28y
4J0B+VUCzS/+kKf21A/MyOLDhpJGmFGNkZpskWNHRJZ65Pq8Pg1ABLFjck4kPOH1U92nQ62px2TZ5yDtAs+HTWUtNqdv+2M/zFkCeryZKGfnto1UbLewX0j+53K11Q+eq3Cq35Z2Pz+Pw
Gw== eyolegoo@gmail.com

user@Godwin MINGW64 ~/CodeTrack (main)
$ |
```

I navigated to New SSH Key by: Go to GitHub → Settings → SSH and GPG keys → Click New SSH Key
I Pasted the copied public key and saved



I tested the SSH credential by running the command `~ ssh -T git@github.com` and got the feedback

```
user@Godwin MINGW64 ~/CodeTrack (main)
$ ssh -T git@github.com
Hi eyolegoo! You've successfully authenticated, but GitHub does not provide shell access.
```

I forked the mini_finance repo using the command ~ `git ls-remote git@github.com:yourusername/mini_finance.git`

Step 2 — Clone Your Fork Locally

Using the SSH option, I cloned the mini_finance repo using the command ~ `git clone git@github.com:yourusername/mini_finance.git`

I navigated to the mini_finance directory using ~ `cd mini_finance`

To list the remote repositories linked to my local git repository (CodeTrack), I ran ~ `git remote -v`. Origin is the remote repository linked to my local repo
Origin points to my forked repo

I added an upstream remote to the original repo using ~ `git remote add upstream https://github.com/pravinmishraaws/mini_finance.git`

NB:: When you fork a repository on GitHub (or GitLab, Bitbucket, etc.), you usually end up with two copies of the repo:

Your fork (on your GitHub account) → You clone this to your local machine.

- Git automatically calls this remote origin.

- You push your own changes here.

The original repository (the one you forked from) → This belongs to someone else.

- To keep track of changes from the original repo, you add it manually as upstream.

```
user@Godwin MINGW64 ~/CodeTrack (main)
$ git ls-remote git@github.com:eyolegoo/mini_finance.git
a8109982d5b97731bd51f52790c3850bf588de91 HEAD
a8109982d5b97731bd51f52790c3850bf588de91 refs/heads/main

user@Godwin MINGW64 ~/CodeTrack (main)
$ git clone git@github.com:eyolegoo/mini_finance.git
Cloning into 'mini_finance'...
remote: Enumerating objects: 61, done.
remote: Counting objects: 100% (61/61), done.
remote: Compressing objects: 100% (56/56), done.
remote: Total 61 (delta 16), reused 44 (delta 5), pack-reused 0 (from 0)
Receiving objects: 100% (61/61), 629.33 KiB | 391.00 KiB/s, done.
Resolving deltas: 100% (16/16), done.

user@Godwin MINGW64 ~/CodeTrack (main)
$ ls -al
total 61
drwxr-xr-x 1 user 197609 0 Aug 28 11:44 ./
drwxr-xr-x 1 user 197609 0 Aug 28 11:25 ../
drwxr-xr-x 1 user 197609 0 Aug 28 01:29 .git/
-rw-r--r-- 1 user 197609 0 Aug 27 11:33 .gitignore
-rw-r--r-- 1 user 197609 287 Aug 28 01:20 contact.html
-rw-r--r-- 1 user 197609 7522 Aug 28 01:20 index.html
drwxr-xr-x 1 user 197609 0 Aug 28 11:45 mini_finance/
-rw-r--r-- 1 user 197609 0 Aug 27 11:33 README.md
-rw-r--r-- 1 user 197609 4180 Aug 27 13:34 style.css

user@Godwin MINGW64 ~/CodeTrack (main)
$ cd mini_finance/

user@Godwin MINGW64 ~/CodeTrack/mini_finance (main)
$ git remote -v
origin git@github.com:eyolegoo/mini_finance.git (fetch)
origin git@github.com:eyolegoo/mini_finance.git (push)

user@Godwin MINGW64 ~/CodeTrack/mini_finance (main)
$ git remote add upstream https://github.com/pravinmishraaws/mini_finance.git

user@Godwin MINGW64 ~/CodeTrack/mini_finance (main)
```

Step 3 — Create a Feature Branch & Make a Change

I created a new branch and switched to it using ~ **git checkout -b feature-readme-update**. This was done in the **/CodeTrack/mini_finance/** directory
I used ~ **ls -la** to view the contents of this directory

```
user@Godwin MINGW64 ~/CodeTrack/mini_finance (main)
$ git checkout -b feature-readme-update
Switched to a new branch 'feature-readme-update'

user@Godwin MINGW64 ~/CodeTrack/mini_finance (feature-readme-update)
$ ls -la
total 189
drwxr-xr-x 1 user 197609  0 Aug 28 11:45 ./
drwxr-xr-x 1 user 197609  0 Aug 28 11:44 ../
drwxr-xr-x 1 user 197609  0 Aug 28 11:50 .git/
-rw-r--r-- 1 user 197609 510 Aug 28 11:45 'ABOUT THIS TEMPLATE.txt'
drwxr-xr-x 1 user 197609  0 Aug 28 11:45 css/
drwxr-xr-x 1 user 197609  0 Aug 28 11:45 fonts/
-rw-r--r-- 1 user 197609 17880 Aug 28 11:45 help-center.html
drwxr-xr-x 1 user 197609  0 Aug 28 11:45 images/
-rw-r--r-- 1 user 197609 32112 Aug 28 11:45 index.html
drwxr-xr-x 1 user 197609  0 Aug 28 11:45 js/
-rw-r--r-- 1 user 197609 19151 Aug 28 11:45 profile.html
-rw-r--r-- 1 user 197609  8282 Aug 28 11:45 README.md
-rw-r--r-- 1 user 197609 22123 Aug 28 11:45 setting.html
-rw-r--r-- 1 user 197609 17018 Aug 28 11:45 transation-detail.html
-rw-r--r-- 1 user 197609 34733 Aug 28 11:45 wallet.html
```

I opened **README.md** using the vim editor ~ **vim README.md** and added a new section, saved and exited:

What was added "This project demonstrates Git operations like clone, pull, push, PR—
a hands-on Mini-Finance tool."

I staged the README.md using ~ **git add README.md**

I saved by committing the staged file using ~ **git commit -m "docs: update README with assignment note"**

```
user@Godwin MINGW64 ~/CodeTrack/mini_finance (feature-readme-update)
$ vim README.md

user@Godwin MINGW64 ~/CodeTrack/mini_finance (feature-readme-update)
$ git add README.md

user@Godwin MINGW64 ~/CodeTrack/mini_finance (feature-readme-update)
$ git commit -m "docs: update README with assionment note"
[feature-readme-update 2b70c4d] docs: update README with assignment note
1 file changed, 2 insertions(+)
```

Step 4 — Pull From Upstream & Push to Origin

I synced changes from upstream's main using the commands:

- ~ **git fetch upstream** this was done on the **feature-readme-update** branch
- ~ **git checkout main** with this command, we swaitched back to the main branch
- ~ **git merge upstream/main** . **NB** the merging was done in the main branch

I switched back to my feature branch using the commands:

- ~ **git checkout feature-readme-update**

~ **git rebase main** . **NB:** is used when you are on a feature branch (not main) and you want to reapply your branch's commits on top of the latest commits from main.

I pushed my branch to my forked using the command:

- ~ **git push -u origin feature-readme-update** . With this my feature branch is available on GitHub under my fork.

```

user@Godwin MINGW64 ~/CodeTrack/mini_finance (feature-readme-update)
$ git fetch upstream
From https://github.com/pravinmishraaws/mini_finance
* [new branch]      main       -> upstream/main

user@Godwin MINGW64 ~/CodeTrack/mini_finance (feature-readme-update)
$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

user@Godwin MINGW64 ~/CodeTrack/mini_finance (main)
$ git merge upstream/main
Already up to date.

user@Godwin MINGW64 ~/CodeTrack/mini_finance (main)
$ git checkout feature-readme-update
Switched to branch 'feature-readme-update'

user@Godwin MINGW64 ~/CodeTrack/mini_finance (feature-readme-update)
$ git rebase main # optional but recommended
Current branch feature-readme-update is up to date.

user@Godwin MINGW64 ~/CodeTrack/mini_finance (feature-readme-update)
$ git push -u origin feature-readme-update
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 432 bytes | 432.00 MiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote:
remote: Create a pull request for 'feature-readme-update' on GitHub by visiting:
remote: https://github.com/eyolego/mini_finance/pull/new/feature-readme-update
remote:
To github.com:eyolego/mini_finance.git
 * [new branch]      feature-readme-update -> feature-readme-update
branch 'feature-readme-update' set up to track 'origin/feature-readme-update'.

user@Godwin MINGW64 ~/CodeTrack/mini_finance (feature-readme-update)

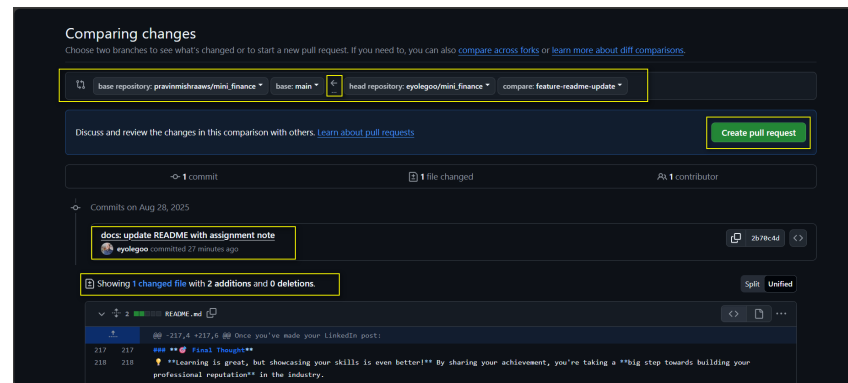
```

Step 5 — Create a Pull Request

Go to your fork on GitHub.

Click Compare & Pull Request.

Make sure it's targeting pravinmishraaws/mini_finance:main from your feature-readme-update branch.

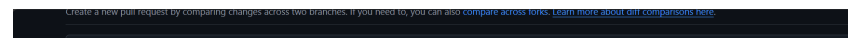


Title: "docs: update README with assignment note"

In the body, add a short description

The short description >> "This PR adds a new section to the README explaining the project's purpose in the context of this GitHub assignment."

Submit the Pull Request.



base repository: pravimishraans/mini_finance • base: main • head repository: eyolegoo/mini_finance • compare: feature-readme-update

 Add a title

docs: update README with assignment note

Helpful resources

GitHub Community Guidelines

Add a description

WritePreview

H B I          

This PR adds a new section to the README explaining the project's purpose in the context of this GitHub assignment.

 Markdown is supported  Paste, drop, or click to add files

 Allow edits by maintainers

Create pull request

docs: update README with assignment note #24

Open

eyolegoo wants to merge 1 commit into [pravimishraans/main](#) from [eyolegoo/feature-readme-update](#)

Conversation

Commits 1

Checks 0

Files changed 1

 eyolegoo commented 1 minute ago

This PR adds a new section to the README explaining the project's purpose in the context of this GitHub assignment



 pravimishraans commented 267ms

 No conflicts with base branch

Changes can be cleanly merged.

Add a comment

WritePreview

H B I          

Reviews

No reviews

Still in progress! [Comment to draft](#)

Assignees

No one assigned

Labels

None yet

Projects

None yet

Milestones